

Collectium — samlet styringsrapport for plattform, moduler, regler og teknisk retning

1. Hoveddefinisjon

Collectium skal bygges som en samlet relasjonsplattform for samlere, objekter, historikk, marked, auksjon, forhandlere, museum, samling, index og administrasjon.

Collectium skal ikke forstås som en vanlig katalog, en vanlig nettbutikk, en vanlig auksjonsside eller en samling med løse enkeltsider. Plattformen skal forstås som ett system der samme objekt, samme relasjon, samme bruker, samme markedshendelse og samme prosess kan brukes på tvers av hele løsningen.

Kjernen i Collectium er:

Objekt

- visningskort
- objektpresentasjon
- relasjonsside
- index / periodeanalyse
- tilbake til filtrert katalog

Dette betyr at ingen del av systemet skal stå alene.

Visningskortet viser objektet kort.

Objektpresentasjonen viser objektet i dybden.

Relasjonssiden forklarer én relasjon i dybden.

Index viser utvikling, marked, periode, trend og analyse på tvers av mange objekter.

Periodefilter binder alt sammen.

Filter Master sørger for at alle sider bruker samme filterkontrakt.

Admin/systemkontroll sørger for at alle sider, knapper, API-ruter, tabeller, views, features og handlinger er sporbare.

2. Overordnet plattformregel

Collectium skal bygges etter denne hovedregelen:

Data og regler eies av database/API/kontrollkjede.
Next.js og React viser, organiserer og håndterer UI.
Template/designmotor styrer visuelt skall.
Frontend skal aldri være sannhet for data, tilgang, priser, filter, relasjoner, auksjon, samling, forhandlerstatus eller medlemskap.

Det betyr:

Database/API/kontroll = sannhet
Next.js = applikasjonsrammeverk, routes, server components og API-ruter
React = komponenter, interaksjon og midlertidig UI-state
Designmotor/template = global layout, visuell identitet og responsivitet
Admin/system = kontroll, logging, status, migrering og deploy-gate

Frontend skal aldri finne opp:

katalogobjekter
filterverdier
relasjoner
relasjonslenker
markedsverdi
brukerstatus
medlemskap
auksjonsstatus
budstatus
forhandlerstatus
tilgang
prisobservasjoner
periodekoblinger

Alt dette skal komme fra API/database/kontrollsystem.

3. Ny teknisk hovedretning

Collectium skal bygges videre som:

Frontend/server: Next.js på Vercel
UI/interaksjon: React-komponenter
Database overgang: MariaDB → Neon Postgres
Filer/bilder: Vercel Blob
Deploy: GitHub → Vercel
Kontroll: Admin/system, sandbox og MariaDB → Neon Control

MariaDB er historisk kilde, kontrollgrunnlag og overgangsarkiv.

Neon skal bli ny hoveddatabase, men bare etter kontroll og godkjenning.

Vercel skal være plattformen for Next.js, API-ruter, server actions, preview, staging, production og app.collectium.no.

Vercel Blob skal brukes til bilder, dokumenter, thumbnails og filvedlegg.

GitHub skal være kodebase, ikke lager for hemmelige nøkler.

Riktig teknisk rollefordeling:

```
GitHub = kode
Vercel = build, deploy, runtime og environment variables
Neon = ny hoveddatabase
Vercel Blob = filer og bilder
MariaDB = overgangskilde og kontrollarkiv
Environment Variables = hemmelige verdier
```

GitHub skal ikke inneholde:

```
DB_PASSWORD
SESSION_SECRET
NEXTAUTH_SECRET
API-nøkler
.env.local
.env.production
ekte databasepassord
private tokens
```

4. DB 4.8 / DB 8.5 / DB 8.6 — databasefaser

Collectium bør forstå databearbeidet i tre praktiske faser:

```
DB 4.8 = eksisterende MariaDB / historisk kilde / kontrollgrunnlag
DB 8.5 = MariaDB → Neon overgang / mapping / kontroll / validering
DB 8.6 = Clean Neon / ny hoveddatabase etter godkjenning
```

DB 4.8 — MariaDB som kilde og kontrollarkiv

MariaDB inneholder eksisterende struktur, tabeller, views, katalogdata, brukerdata, regler, routes, features og historikk.

MariaDB skal ikke kastes før Neon er validert.

MariaDB skal brukes til å kontrollere:

```
eksisterende tabeller  
eksisterende views  
kolonner  
katalogkilder  
brukere  
session  
features  
access  
action-routes  
katalogobjekter  
relasjoner  
prisdata  
historikk  
adminstatus
```

DB 8.5 — overgang fra MariaDB til Neon

DB 8.5 er overgangsfasen. Her skal systemet kontrollere og mappe:

```
tabeller  
views  
felter  
ID-er  
object_id  
source_key  
object_group  
relation_type  
relation_key  
feature_key  
page_key  
action_route  
read_view  
write_table  
log_category  
log_action
```

Ingen kilddata skal flyttes ukritisk.

Hovedregelen for migrering:

1. Struktur først
2. Regler / metoder / prosesser etterpå
3. Kilddata til slutt

DB 8.6 — Clean Neon

DB 8.6 er ønsket fremtidig hoveddatabase.

Neon kan først bli sann hoveddatabase når kontrollen viser OK på:

```
struktur
tabeller
views
felter
ID-mapping
katalognøkler
source_key
object_group
relasjoner
Filter Master
periodefilter
DB-kontrollkjede
auth/session
brukere
medlemskap
samling
forhandler
auksjon
admin/action-routes
sidekrav
innholdskrav
API-ruter
logging
datakvalitet
kildedata
```

5. Låst kontrollkjede

Alle viktige sider, knapper, handler og API-ruter skal følge samme kontrollmodell.

Låst kjede:

```
ct_app_pages
→ ct_app_page_features
→ ct_app_features
→ ct_feature_access_rules
→ ct_v_feature_access_resolved
→ ct_feature_action_routes
→ API
→ table/view
→ logg
```

I Next.js/Neon-retning kan den forstås slik:

```
page_key
→ feature_key
→ access_rule
→ action_route
→ API route
→ registry / table / view
→ log_category / log_action
```

Ingen knapp skal være løs.

En knapp skal være én av tre typer:

1. Koblet til feature_key + API/server action
2. Deaktivert med forklaring fra tilgangsregel
3. Ren lokal UI/template-kontroll

Eksempel:

```
Knapp: Legg til ønskeliste
feature_key: collection.wishlist.toggle
API: /api/collection/wishlist/toggle
write_table: user/object state
logging: wishlist.toggle
```

6. Teknisk dataflyt

Ny standard dataflyt:

```
Neon / MariaDB
→ DB-kontroll / feature / access / action-route
→ API / backend / server action
→ Next.js route / server component
→ React component
→ UI
→ logging
```

React skal ikke koble direkte til database.

React-komponenter skal heller ikke finne opp data, priser, filtrering, auksjonsstatus, relasjoner eller brukerstatus.

All handling skal kunne spores tilbake til:

```
side
route
page_key
feature_key
access_rule
action_route
API-route
read_view
write_table
log_category
log_action
```

7. Ansvarsdeling i Next.js / React / API / database

Next.js har ansvar for

```
routing
layouts
server components
route handlers
server actions
metadata
loading/error/not-found
server-side datahenting
auth/session-kontroll
API-lag
rendering
```

React har ansvar for

```
komponenter
visning
interaksjon
midlertidig UI-state
segmentbytte
visningsbytte
filterpanel
knapper
modaler
tabs
empty/loading/error states
```

API/backend har ansvar for

```
inputvalidering  
session  
rolle  
medlemskap  
feature-check  
access-check  
action-route-check  
read/write  
logging  
standard JSON-respons  
feilhåndtering  
datakontrakter
```

Database har ansvar for

```
sannhet  
katalogdata  
brukerdata  
medlemskap  
features/brytere  
tilgang  
routes  
relasjoner  
markedsdata  
samling  
auksjon  
forhandler  
logging
```

8. Filstruktur og prosjektstandard

Ny hovedstruktur bør være:

```
app/  
  layout.tsx  
  page.tsx  
  
katalog/  
  page.tsx  
  [sourceKey]/  
    page.tsx  
  
objekt/  
  [sourceKey]/
```



```
[objectGroup]/
  [objectId]/
    page.tsx

relasjon/
  [relationType]/
    [relationKey]/
      page.tsx

index/
  page.tsx

min-side/
  page.tsx

samling/
  page.tsx

auksjon/
  page.tsx

nettbutikk/
  page.tsx

forhandler/
  page.tsx

admin/
  page.tsx
  system/
    mariadb-neon/
      page.tsx
    db84/
      page.tsx
    filter-master/
      page.tsx
    period-filter/
      page.tsx
    unit-test/
      page.tsx

api/
  auth/
  catalog/
  object/
  relations/
  index/
  collection/
  auction/
  shop/
  dealer/
```

```
admin/  
system/  
  
components/  
  layout/  
  templates/  
  ui/  
  catalog/  
  object/  
  relations/  
  index/  
  collection/  
  auction/  
  shop/  
  dealer/  
  admin/  
  
lib/  
  db/  
  auth/  
  access/  
  api/  
  logging/  
  formatters/  
  mappers/  
  types/  
  recovery/  
  
styles/  
  globals.css  
  collectium-theme.css  
  
public/  
  images/  
  logos/  
  icons/  
  
docs/  
scripts/  
tests/
```

Vanlige sider skal ikke eie design.

Vanlige sider skal bare levere:

```
innhold  
data  
komponenter  
handler  
empty state
```

```
loading state
error state
```

Globalt design skal ligge i:

```
components/layout/
components/templates/
styles/
global tokens
AppShell
Topbar
Sidebar
MobileMenu
PageFrame
WorkspaceLanes
CollectiumSignature
```

9. Kode-, fil-, endrings- og loggregel

Alle nye hovedfiler skal ha dokumentasjonsheader.

Dette gjelder spesielt:

```
page.tsx
layout.tsx
route.ts
server actions
React-komponenter
API-klienter
DB-queryfiler
auth/access-filer
admin/systemfiler
importscripts
recoveryscripts
testfiler
```

Standardfil må dokumentere:

```
overskrift
definering / formål
bruksområde
berørte sider / routes
berørte feature_keys
berørte API-ruter
berørte tabeller/views
```

```
dataretning
logging
versjon
endringsregel
```

Hovedkode skal ikke overskrives direkte uten snapshot, manifest og godkjenning.

Kjernefiler:

```
app/layout.tsx
app/page.tsx
components/layout/*
components/templates/*
styles/globals.css
lib/db/*
lib/auth/*
lib/access/*
lib/api-response/*
lib/logging/*
lib/recovery/*
app/api/admin/system/*
```

Endringer skal gjøres som:

```
ny modul
ny komponent
ny helper
kontrollert patch
ny versjonert fil
feature branch
recovery snapshot
manifestoppføring
endringslogg
```

10. Designmotor og template

Designmotoren er det globale visuelle systemet.

Den styrer:

```
layout
skin
farger
typografi
responsivitet
```

```
menyer
kort
paneler
arkivfaner
filterpanel
signaturhjørner
Collectium-identitet
spacing
radius
shadows
visningsmodus
```

Designmotoren skal ikke eie data.

Riktig skille:

```
Database/API/kontroll = sannhet og regler
Designmotor = visuelt skall og uttrykk
React = komponenter og interaksjon
```

Vanlige sider skal ikke lage:

```
egen topbar
egen sidebar
egen bakgrunn
egen shell-layout
egne panelrammer
egne globale shadows
lokal designmotor
lokal skin-state
egen layoutlogikk
```

Dette skal ligge globalt.

11. Skins og visuell retning

Collectium skal ha kontrollerte skins, men skin skal aldri endre datastruktur eller systemlogikk.

Gjeldende skin-retning:

```
Collectium
Samler / Enkel
Museum
Finans
```

I teknisk test kan dette kontrolleres som:

```
Collectium Signature Light
Collectium Signature Dark
Collectium Minimal Light
Collectium Minimal Dark
```

Viktig regel:

```
Skin kan endre farge, kontrast, typografi, stemning og visuell signatur.
Skin skal aldri endre data, rekkefølge, API-logikk, tilgang eller
komponentansvar.
```

Museum er ikke bare en visuell dekor. Museum skal støtte historisk presentasjon, relasjoner, perioder, regenter, personer, funn, proveniens og kulturell kontekst.

Finans skal være tydelig annerledes enn Museum og Collectium. Finans skal ha et mer analytisk uttrykk med markedsdata, verdier, grafer, trend, utvikling og sammenligning.

12. Responsivitet og arbeidsflater

Responsivitet skal styres globalt.

Ingen vanlig side skal løse mobil, tablet, desktop eller bredskjerm på egen måte.

Hovedmoduser:

```
Mobil
Tablet
Desktop
Bredskjerm
TV / presentasjon
```

Mobil

```
0-719px
sticky toppbar
ingen fast venstre sidemeny
filter som overlay
én kolonne
store touchfelt
mobilmeny
søk / aktivitet / meny tilgjengelig
```

Tablet

720–1100px
filter over resultater
1–2 kolonner
kompakt panel
foldbare seksjoner

Desktop

1101–1899px
normal Collectium-layout
sidemeny
topbar
ett hovedinnholdsområde
kontrollert maks bredde

Bredskjerm

1900px+
sideveis arbeidsflate
flere lanes
sidemeny 200–300px
innholdsbaner rundt 1000px
filter/resultat/objekt/relasjon/finans kan ligge side om side

TV / presentasjon

2900px+
presentasjonsmodus
større typografi
færre aktive arbeidsbaner
ikke samme som bredskjerm arbeidsflate

Bredskjerm betyr arbeidsflate.

TV betyr presentasjon.

De skal ikke blandes.

13. Collectium som relasjonsplattform

Collectium skal bygges som en relasjonsplattform, ikke som en flat produktliste.

Et objekt skal aldri bare være en katalograd.

Et objekt kan være:

seddel
mynt
medalje
verdipapir
kunstobjekt
antikvitett
dokument
militaria
historisk gjenstand
andre samleobjekter

Et objekt kan:

vises i katalogen
vises som visningskort
åpnes i objektpresentasjon
kobles til relasjonssider
eies av en samler
legges i ønskeliste
legges i favoritter
legges i samling
sendes til forhandler
legges ut i auksjon
selges i nettbutikk
bytte eier
få dokumentasjon
få proveniens
få prisobservasjoner
inngå i index
inngå i historisk analyse
inngå i museumvisning

Objektet skal alltid kunne spores tilbake til sin katalogidentitet.

14. Katalogmodulen

Katalogen er Collectium sin felles datakjerne.

Den skal ikke være en flat produktliste.

Katalogen skal vise:

objekt
kilde


```
objektgruppe  
opprinnelse  
produsent  
utgave  
periode  
personer  
signaturer  
historiske relasjoner  
materiale  
marked  
brukerrelasjon  
samling  
verdi  
trend
```

Objekter skal alltid identifiseres med:

```
object_id + object_group + source_key
```

For Norske sedler:

```
source_key = norske_sedler  
object_group = banknote
```

Filter skal alltid være source-scoped:

```
source_key + object_group + filter_field + filter_value
```

Dette hindrer at sedler, mynter og andre kilder blandes i frontend.

Katalogen skal støtte segmentene:

```
Samler  
Historie  
Finans
```

Katalogen skal støtte visninger:

```
horizontal  
standing  
list  
card  
table
```

```
museum  
slide
```

Visning skal styres med:

```
data-view="horizontal|standing|list|card|table|museum|slide"
```

Ikke med separate hardkodede sider.

15. Filter Master

Filter er ikke bare katalogfilter.

Filter Master er en felles plattformmotor.

Den skal brukes i:

```
katalog  
index  
objektpresentasjon  
relasjonspresentasjon  
auksjon  
nettbutikk  
forhandler  
min samling  
marked/finans  
transaksjoner  
admin  
systemkontroll
```

Filterstruktur:

```
Filter Master  
→ samletypedekkende filter  
→ objektspesifikt filter  
→ periodefilter enkel  
→ periodefilter avansert  
→ side-/prosessespesifikke filter
```

Filter Master skal hindre at hver side lager sin egen filterlogikk.

Filter Master skal kunne håndtere:

source_key
object_group
land
kilde
objekttype
segment
tilgangsnivå
markedskanal
samling
auksjon
nettbutikk
forhandler
periode
relasjon
brukerstatus
datakvalitet

16. Objektspesifikt filter

For sedler skal objektfilteret følge denne rekkefølgen:

Valør / objektbetegnelse
→ Årstall
→ Litra / nummer / detalj
→ Valørutgave / serie
→ Variant / type
→ Signatur / personer
→ Konge / regent / dynasti

Feltmapping:

denomination_raw_no	→ Valør / objektbetegnelse
object_year_label	→ Årstall
litra_raw_no	→ Litra / nummer / detalj
denomination_issue_raw_no	→ Valørutgave / serie
variant_type_raw_no	→ Variant / type
signature_raw_no	→ Signatur / personer
ruler_name_raw_no	→ Konge / regent
historical_ruler_raw_no	→ Historisk regent
dynasty / house	→ Dynasti / kongehus

Viktig korrigering:

denomination_issue_raw_no skal vises som Valørutgave / serie, ikke bare Utgave.

Det skal brukes i:

```
katalogfilter  
visningskort  
objektpresentasjon  
AI-søk  
relasjonssøk  
resolved views  
index  
periodefilter
```

17. Periodefilter

Periodefilter er tidsmotoren i Collectium.

Periodefilter skal ikke være et lite lokalt filter inne i visningskortet.

Periodefilter skal være felles plattformmotor for:

```
/katalog  
/index  
/objekt/[sourceKey]/[objectGroup]/[objectId]  
/relasjon/[type]/[slug]  
/auksjon  
/nettbutikk  
/forhandler  
/min-side/samling  
/admin/catalog  
/admin/system/migration
```

Periodefilter skal ikke bare filtrere etter år.

Det skal kunne koble flere tidslag:

```
publiseringsår  
objektår  
utgaveperiode  
produksjonsperiode  
regentperiode  
dynasti / maktstruktur  
historisk periode  
historisk hendelse
```

funnperiode
marked/performance-periode
auksjonsperiode
samlingstransaksjonsperiode
indexperiode

Enkel periodefilter

Tilgjengelig for alle under Sølv.

Kan inneholde:

år fra
år til
periode
århundre
historisk hovedperiode
regent / konge
før / etter

Eksempler:

1800–1899
1900–1945
Etter 1945
Oscar II
Haakon VII
Norges Bank periode

Avansert periodefilter

Krever Sølv eller høyere.

Skal være både:

et panel inne i andre sider
en egen side

Egne sider:

/filter/periode
/filter/periode/avansert

Avansert periodefilter kan kombinere:

publiseringsår
objektår
utgaveperiode
produksjonsperiode
regentperiode
dynasti / maktstruktur
historisk periode
historisk hendelse
funnperiode
marked/performance-periode
auksjonsperiode
samlingstransaksjonsperiode
indexperiode

18. Visningskort UI 8.5

Visningskortet er inngangen fra katalog, samling, relasjon, index, nettbutikk eller auksjon inn til objektpresentasjonen.

Visningskortet er objektets kompakte presentasjon.

Alle visningskort skal bruke samme datastruktur og samme markup.

Skin kan endre farge og uttrykk, men ikke data, rekkefølge eller systemlogikk.

Visningskort kan brukes i:

Katalog
Samling
Nettbutikk
Auksjon
Relasjonspresentasjon
Index
Søk
Favoritter
Ønskeliste

Hovedinformasjon på visningskort skal alltid vises øverst og bestå av:

Overskrift
Årstall
Valør
Utgave / serie

Utgave
Variant

Disse feltene utgjør objektets primære identitet.

Felles innhold:

Objektbilde / seddelflate
Tittel
Valør
År
Utgave
Variant
Sjeldenhet
Signatur
Konge / regent
Kilde
object_group
source_key
katalognummer
Hjerte
Stjerne
Auksjon
Nettbutikk
Estimert pris
Vurdert-status
Trend
Trendperiode
Åpne objekt
Se relasjon
Legg i samling
Collectium signaturhjørne

Visningskort skal alltid kunne åpne:

Objektpresentasjon
Relasjonspresentasjon
Samling
Nettbutikkobjekt
Auksjonsobjekt

Segmenttilpasning

Samler:

hjerte
stjerne

legg i samling
ønskeliste
favoritt
kvalitet
sjeldenhet
brukerstatus

Historie:

år
utgave
regent / konge
signatur
historisk periode
materiale
relasjoner

Finans:

estimert verdi
trend
trend %
likviditet
auksjonsstatus
nettbutikkstatus
prisobservasjoner

19. Visningskort — handlinger

Visningskortet skal ha tre hovedhandlingar:

Åpne objekt
Relasjoner
Legg i samling

Åpne objekt

Åpner full objektpresentasjon.

/objekt/[sourceKey]/[objectGroup]/[objectId]

Eksempel:

/objekt/norske_sedler/banknote/1459

Relasjoner

“Se relasjon” skal ikke være en løs knapp.

Den skal åpne en liste eller meny med objektets faktiske relasjoner.

Eksempel:

År · 1877
Regent · Oscar II
Signatur · Winge / Getz
Motiv · Riksvåpen
Utgave · 1. utgave
Kilde · Norske sedler
Valør · 100 kroner
Produsent · Norges Bank

Klikk på en relasjon åpner relasjonspresentasjonen.

Legg i samling

Skal kobles til brukerens samling og brukerstatus.

Det skal ikke bare være lokal frontend-state.

20. Objektpresentasjon

Objektpresentasjonen er full visning av ett objekt.

Den skal ikke være et større katalogkort.

Den skal forklare objektet som:

katalogpost
samlersobjekt
historisk objekt
finansielt objekt
relasjonsnode
markedsobjekt

Route:

```
/objekt/[sourceKey]/[objectGroup]/[objectId]
```

Eksempel:

```
/objekt/norske_sedler/banknote/1459
```

Slug kan legges til senere, men slug er aldri teknisk sannhet.

Teknisk oppslag er alltid:

```
source_key + object_group + object_id
```

Objektpresentasjonen skal bygges rundt tre hovedsegmenter:

```
Samler  
Historie  
Finans
```

Den skal være koblet til:

```
Master Filter  
Objektfilter  
Auksjonsfilter  
Nettbutikkfilter  
Periodefilter  
Relasjonspresentasjoner  
Samling  
Katalog  
Index
```

21. Objektpresentasjon — innhold

Objekt-hero

```
hovedbilde  
bildegalleri  
fallback: Bilde ikke registrert  
tittel  
katalognummer  
kilde  
object_group
```

source_key
object_id

Identitet

Valør / objektbetegnelse
Årstall
Litra / nummer / detalj
Valørutgave / serie
Variant / type
Signatur / personer
Konge / regent / dynasti

Samler

hjerter
stjerne
i min samling
egne lister
kvalitet
sjeldenhet
kjøpspris hvis bruker eier objektet
kjøpsdato
kjøpt fra
solgt til
notater
private bilder
deling
dokumentasjon

Historie

produsent / utsteder
utgave / serie
utgaveperiode
publiseringsår
objektår
regent / konge
regentperiode
dynasti / maktstruktur
signatur / personer
motiv
materiale
historisk periode
historiske hendelser
funn

proveniens
relasjoner

Finans

estimert verdi
fallback: Ikke vurdert
trend
trend %
trendperiode
likviditet
siste prisobservasjon
auksjonsstatus
nettbutikkstatus
markedsgrunnlag
indexkobling

Viktig regel:

0 kr / 0.00 betyr manglende verdi.
Det skal aldri tolkes som ekte markedspris.

Hvis verdi mangler, skal det vises:

Ikke vurdert
Mangler markedsverdi
Henter datagrunnlag

22. Objektpresentasjon — relasjon og periode

Objektpresentasjonen skal alltid ha periode- og relasjonskobling.

På objektpresentasjon skal periodefilter ikke primært være et filterpanel. Det skal også være kontekst.

Objektpresentasjonen skal vise hvordan objektet hører til tidsmessig:

Objektår → relasjon til år
Publiseringsår → relasjon til år
Utgaveperiode → relasjon til periode
Regentperiode → relasjon til konge/regent
Historisk periode → relasjonspresentasjon
Markedperiode → index/finans

Eksempel:

1 krone 1917 Litra A

År: 1917

→ /relasjon/ar/1917

Regent: Haakon VII

→ /relasjon/regent/haakon-vii

Historisk periode: Første verdenskrig

→ /relasjon/periode/forste-verdenskrig

Marked 12 mnd

→ /index?object_group=banknote&trend_period=12m

Objektpresentasjonen skal ha egen fane:

Relasjon til andre objekttyper

Denne fanen skal vise koblinger til:

andre samleobjekter
andre objektgrupper
perioder
personer
motiver
hendelser
kilder
produsenter
regenter
historiske relasjoner
finansielle relasjoner

23. Relasjonspresentasjon

Relasjon er ikke bare databasekobling.

Relasjon er også frontend-navigasjon.

Når en kjent verdi vises, skal den kunne bli klikkbar:

årstall
konge/regent
person/signatur
motiv
produsent

```
kilde
utgave
valør
historisk periode
materiale
funn
proveniens
finansiell kontekst
```

Route:

```
/relasjon/[type]/[slug]
```

Eller teknisk:

```
/relasjon/[relationType]/[relationKey]
```

Eksempler:

```
/relasjon/regent/oscar-ii
/relasjon/regent/olav-v
/relasjon/ar/1980
/relasjon/person/winge-getz
/relasjon/motiv/riksvaapen
/relasjon/kilde/norske-sedler
/relasjon/produsent/norges-bank
/relasjon/utgave/1-utgave-1877
/relasjon/valor/100-kroner
/relasjon/periode/unionstid
```

Relasjonspresentasjon skal vise:

```
relasjonstype
navn / label
slug
periode
kort forklaring
historisk kontekst
finansiell kontekst der relevant
bilder / portrett / heraldikk / signaturbilde der relevant
relaterte objekter
relaterte objektgrupper
relaterte perioder
relaterte hendelser
relasjoner til andre relasjoner
lenke til filtrert katalog
```

lenke til indexanalyse
lenke til samling
lenke til nettbutikk
lenke til auksjon

Relasjonssiden skal ha egen fane:

Relasjon til andre samleobjekter

Denne fanen skal vise koblinger på tvers av samletyper og relasjoner.

Eksempel:

Kong Olav V
→ motiv
→ person
→ regentperiode
→ historisk periode
→ sedler
→ mynter
→ medaljer
→ dokumenter
→ andre samleobjekter

24. Relasjonspresentasjon — periodekontekst

Ingen relasjonsside uten periodekontekst.

Eksempel for regent:

/relasjon/regent/oscar-ii

Viser:
Oscar II
Regentperiode
Union
Historisk periode
Objekter publisert i perioden
Objekter med motiv/personkobling
Sedler
Mynter
Hendelser
Markedskontekst

Eksempel for år:

/relasjon/ar/1980

Viser:

År 1980

Historisk kontekst

Finansiell kontekst

Objekter publisert i 1980

Auksjonsdata

Markedsdata

Indexkobling

Eksempel for periode:

/relasjon/periode/unionstid

Viser:

periode

regenter

objekter

historiske hendelser

samfunnskontekst

marked

index

25. Index / marked / finans

Index er ikke vanlig forside.

Index er:

markedsindex

analyseforside

utviklingsdashboard

periodeanalyse

relasjonsanalyse

finansmotor

Index skal bruke data fra:

katalog

objekter

relasjoner

samling

auksjon

nettbutikk


```
marked
transaksjoner
prisobservasjoner
historiske hendelser
finansdata
```

Index skal svare på spørsmål som:

```
Hvilke objekter utvikler seg best?
Hvilke perioder har høyest aktivitet?
Hvilke regenter eller historiske perioder har best verdiutvikling?
Hvilke objektgrupper er mest likvide?
Hvordan utvikler samlingen seg mot markedet?
Hvordan utvikler sedler, mynter eller andre samletyper seg over tid?
Hvilke objekter mangler verdi?
Hvilke objekter har høy aktivitet?
Hvilke kilder har best datakvalitet?
```

Index skal ha periodefilter.

Index skal kunne filtrere på:

```
år fra
år til
periode
århundre
regent
dynasti
historisk periode
historisk hendelse
source_key
object_group
land
kilde
valør
materiale
markedskanal
trendperiode
verdiområde
datakvalitet
```

Index skal kunne åpne:

```
filtrert katalog
relasjonsside
objektpresentasjon
markedsgraf
```

periodeanalyse
samlersanalyse
auksjonsanalyse

26. Markedsverdi og finansprosess

Collectium skal bygge markedsverdi fra ekte prisobservasjoner.

Ikke fra demo-tall.

Datagrunnlag kan være:

market values
market sales
price observations
auction lots
collection transactions
dealer objects
catalog market summary
nettbutikkpriser
historiske salg

Markedsverdi bør bygges fra:

siste pris
+ tre siste relevante priser
+ kvalitet/grade
+ valuta
+ kilde
+ markedskanal
+ observasjonsdato

Finanssegmentet skal kunne vise:

markedsverdi
trend
trend %
likviditet
auksjonsresultater
nettbutikkpriser
transaksjoner
gull/sølv/metallverdi
valuta
inflasjon
børs/index-sammenligning

fortjeneste
utvikling over tid

Regel:

0.00 = mangler verdi
0.00 er ikke ekte markedspris

27. Bruker / Min side

Min side er brukerens private kontrollsenter.

Den skal være rollebasert.

Vanlig bruker ser brukerfunksjoner.

Forhandler får ekstra forhandlerflate.

Admin får adminflate.

Min side skal inneholde:

Oversikt
Profil
Medlemskap
Min samling
Ønskeliste
Favoritter
Kjøp / salg
Transaksjonslogg
Prosesser
Varsler
Meldinger
Dokumenter
Innstillinger
Sikkerhet

Brukeren skal kunne:

logge inn
se medlemskap
endre profil
se samlingsstatus
markere hjerte
markere stjerne

legge objekt i Min samling
registrere kjøp
registrere salg
laste opp dokumentasjon
følge auksjon
se varsler
se meldinger
dele objekt eller gruppe med tidslenke
sammenligne egen samling med markedet

Min side skal vise:

egne objekter
aktive bud
aktive kjøp
historiske kjøp
historiske salg
pågående prosesser
varsler
meldinger
betalingsstatus
medlemsstatus
forhandlerstatus hvis relevant
adminstatus hvis relevant

28. Brukertyper og medlemskap

Collectium bør ha disse brukertypene:

1. Gjest
2. Registrert bruker
3. Samler
4. Betalt medlem
5. Forhandler
6. Admin
7. Superadmin / Collectium

Gjest

Kan se:

offentlig katalogutdrag
noen objekter
begrenset historisk informasjon
begrenset markedsinformasjon
landingsside

medlemskap/priser
offentlige auksjoner / nettbutikk etter regler

Kan ikke:

lagre objekter
ha egen samling
se full historikk
se full finansdata
by på auksjon
kjøpe uten nødvendig registrering
se private samlinger

Registrert bruker

Kan:

logge inn
ha profil
lagre grunnleggende preferanser
se begrenset katalog
markere enkelte objekter
starte samling med begrenset funksjon
motta varsler
se medlemskapstilbud

Samler

Kan:

legge objekt i Min samling
bruke hjerte / ønskeliste
bruke stjerne / favoritt
lage egne samlingslister
registrere kjøp
registrere salg
registrere kjøpspris
registrere kjøpsdato
registrere kjøpt fra
registrere solgt til
legge inn notater
dele objekt eller gruppe med tidslenke
følge objekter
følge auksjoner
sammenligne egen samling med markedet

Betalt medlem

Medlemskap styrer:

hvor mange filter brukeren får se
hvor mye historikk brukeren får se
hvor mye finansdata brukeren får se
hvor mange land/kilder brukeren får tilgang til
om brukeren får avansert index
om brukeren får eksport
om brukeren får full markedsanalyse

Nivåer:

Free
Bronze
Silver
Gold
Platinum

Tilgangsmodell for filter og periode

Free / Bronze
→ Filter Master basis
→ enkel periodefilter
→ begrenset objektspesifikt filter

Silver
→ avansert periodefilter
→ flere historiske filter
→ flere marked/index-filter

Gold
→ forhandler-, auksjon- og markedskoblinger
→ dypere objekt- og handelsfilter

Platinum
→ alle land
→ alle samletyper
→ full relasjons-, periode- og finansfiltrering

Budgivning

Alle brukere må være pålogget for å kunne gi bud.

Brukeren må også ha medlemskap og tilgang som gir rett til:

```
objektpresentasjon
relasjonssider
periodefilter
auksjonsobjekt
budfunksjon
```

29. Samling

Samling er ikke bare en liste.

Samling er brukerens private eller delbare samlerområde.

Samling skal støtte:

```
i_min_samling
ønskeliste / hjerte
favoritt / stjerne
egne lister
kjøp
salg
transaksjoner
dokumentasjon
private bilder
deling
proveniens
verdiutvikling
samleranalyse
```

Brukerens konkrete eksemplar skal ikke forveksles med katalogobjektet.

Riktig modell:

```
Katalogobjekt = seddeltypen / mynttypen / objektdefinisjonen
Brukerobjekt = brukerens konkrete eksemplar
```

Brukerobjekt kan ha egne data:

```
kvalitet
bilder
kjøpspris
kjøpsdato
kjøpt fra
notater
sertifikat
```

```
eierstatus  
private proveniensnotater
```

30. Deling

Brukere skal kunne dele:

```
ett objekt  
en gruppe objekter  
en samlingsliste  
et objektkort  
en historisk visning
```

Deling bør være tidsstyrt:

```
8 timer  
12 timer  
18 timer  
24 timer  
48 timer
```

Deling skal bruke:

```
share_token  
tilgangsvindu  
besøkslogg  
objektkobling  
eierkontroll  
samtykke
```

Hvis mottaker ikke er medlem, kan det vises tilbud etter regler.

Private proveniens- og eierdata skal ikke vises uten samtykke.

31. Proveniens

Proveniens skal være en strukturert historie-, verdi- og relasjonsmodul.

Ikke bare et tekstfelt.

Proveniens må skille mellom:

privat / brukerlagt eierskapsproveniens
offentlig / historisk / kjent proveniens

Privat proveniens eies/kontrolleres av personen som la den inn og personen/parten den gjelder.

Forhandler, selger, andre brukere eller offentligheten skal ikke se identitet, eierskap, transaksjon, verdi eller relaterte objekter uten godkjenning.

Offentlig proveniens kan være:

skipsvrak
sunken boat
Vikinggrav
skattefunn
museumskontekst
kjent samling
historisk eier
offentlig auksjonshistorikk

Proveniens bør støtte:

år / periode
dato-presisjon
estimert pris
transaksjonspris
omsetningsbeløp
valuta
antall objekter
transaksjonstype
relaterte objekter
proveniensgruppe
kilde
synlighet
samtykke

32. Forhandler

Forhandler er en profesjonell rolle og mellomledd mellom bruker/eier og marked.

Forhandler skal kunne:

søke forhandlerstatus
dokumentere firma/person
dokumentere tillatelser og kategorier
akseptere forhandleravtale

få godkjente objektgrupper
motta objekt fra bruker
kontrollere objektdata
kontrollere bilder, kvalitet og ekthet
foreslå pris
foreslå utrop
foreslå salgsstrategi
sende forslag til bruker
legge objekt til auksjon etter brukerens godkjenning
legge objekt i nettbutikk etter regler
håndtere bud/salg
gjennomføre oppgjør
rapportere gebyrer og provisjon
se egne avtaler
se egne prosesser

Forhandlerprosessen:

forhandler registrerer/søker tilgang
→ admin godkjenner
→ forhandler får objektgruppetilgang
→ objekt mottas/registreres
→ objekt vurderes
→ objekt settes til auksjon, nettbutikk eller begge
→ salg/bud gjennomføres
→ gebyr/fee beregnes
→ oppgjør registreres

Forhandlerobjekt skal behandles som egen status/kobling, separat fra brukerens private samling.

Viktig regel:

Forhandler skal ikke sette Collectium-fee.
Collectium-fee settes av admin/Collectium sentralt.
Forhandler-fee kan ligge per objektlinje.

33. Forhandlergodkjenning

Forhandler må gjennom godkjenningsflyt.

Steg 1: Forhandler registrerer seg

Forhandler må oppgi:

firmanavn
organisasjonsnummer
kontaktperson
adresse
land
telefon
epost
nettside
betalingsinformasjon
konto for oppgjør
MVA-status

Steg 2: Forhandler søker om godkjenning

Forhandler må sende dokumentasjon:

firmaattest
identitet / kontaktperson
brukthandelbevilling hvis relevant
kategori-/objektgruppetilgang
forsikringsinformasjon
betalings-/oppgjørsinformasjon
aksept av Collectium-avtale

Steg 3: Admin godkjenner forhandler

Admin kontrollerer:

forhandlerdata
dokumentasjon
kategori/objektgruppe
rettigheter
avtale
fee-oppsett
risiko/status

Steg 4: Forhandler får objektgruppetilgang

Forhandler skal bare kunne håndtere objektgrupper de er godkjent for.

Eksempler:

sedler
mynter
verdibrev
kunst
antikviteter

historiske dokumenter
reklameskilt

Frimerker skal ikke være ønsket hovedgruppe i Collectium-retningen. Verdibrev / securities skal med.

34. Auksjon

Auksjon er markedskanal på tvers av forhandlere.

Auksjon skal håndtere:

objektpublisering
budgivning
budhistorikk
vinner
betaling
frister
reserve/nestebudgiver
oppgjør
tvist
historikk
prisobservasjon
markedsobservasjon

Grunnflyt:

forhandler legger objekt til auksjon
→ objekt får auksjonsstatus
→ bruker kan følge objekt
→ bruker kan by
→ bud registreres
→ auksjon avsluttes
→ vinner får betalingsfrist
→ betaling/oppgjør håndteres
→ resultat blir markedsobservasjon
→ verdi/trend/index oppdateres

Auksjon skal være eget filter på tvers av alle forhandlere:

market_channel = auction
dealer_id = *

35. Budgiver 2 / reservekjøper

Ved auksjon skal budgiver 2 kunne defineres som reservekjøper dersom vinner ikke betaler innen avtalt frist.

Budgiver 2 må informeres om rettigheter og plikter før eller under budgivning.

Regel:

Hvis vinner ikke betaler innen betalingsfrist,
kan budgiver 2 få tilbud om kjøp til høyeste gyldige bud etter definerte
regler.

Dersom budgiver 2 avslår, går objektet tilbake til forhandler/eier.

Forhandler/eier kan da vurdere:

ny auksjon
nettbutikk
direktesalg
tilbaketrekking
ny vurdering

Denne prosessen må vises i:

brukerflyt
forhandlerflyt
auksjonsflyt
adminflyt
transaksjonslogg
varselssystem
prosesslogg

36. Nettbutikk

Nettbutikk viser objekter som kan kjøpes direkte.

Nettbutikk skal håndtere:

butikk
lagerstatus
pris
land
levering

```
forhandler
kjøp
kontakt
betaling
overføring til samling
markedsobservasjon
```

Flyt:

```
forhandler legger objekt i nettbutikk
→ objekt får shop-status
→ pris og lagerstatus vises
→ bruker kan kjøpe eller kontakte forhandler
→ salget kan gi markedsobservasjon
→ objekt kan flyttes til samling/transaksjon
```

Nettbutikk skal bruke samme Filter Master og periodefilter der objekter, marked, kilde, periode og samling er relevant.

37. Transaksjonslogg

Transaksjonsloggen skal samle alt som skjer økonomisk eller prosessmessig rundt et objekt.

Transaksjonsloggen skal inneholde:

```
kjøp
salg
bud
auksjonsresultat
betaling
oppgjør
refusjon
gebyr
forhandlerkostnad
Collectium-fee
provisjon
dokumentasjon
statusendring
```

Felt:

```
dato
objekt
kilde
object_group
source_key
```

- transaksjonstype
- motpart
- forhandler
- auksjon
- pris
- valuta
- betalingsstatus
- oppgjørstatus
- dokumenter
- notater
- loggstatus

Transaksjonsloggen skal kobles til:

- Min side
- Samling
- Forhandler
- Auksjon
- Nettbutikk
- Index
- Finanssegment
- Admin

38. Museum / Historie

Museum er ikke bare en skin.

Museum er en historisk presentasjonsflate bygget på relasjonsdata.

Museum-/historieområdet skal vise:

- historiske perioder
- regenter/konger
- dynasti / maktstruktur
- lokale/regional herskere
- personer
- signaturer
- produsenter
- materiale
- funn
- proveniens
- territorier
- historiske hendelser
- relaterte objekter
- samfunnskontekst

Museum view i katalog skal vise objektet med relasjons- og kontekstpanel.

Ikke bare et vanlig kort.

Viktig regel:

Historie skal ikke forenkle alt til én nasjonal konge.
Systemet må støtte lokale/regional herskere, territorier, dynastier og flere historiske lag.

Mini digitalt museum skal kunne brukes for:

kommuner
lokalsamfunn
private samlinger
unike objekter
historiske funn
museumsobjekter
lokale utstедelser
objekter som ikke finnes i vanlige kataloger

39. Historisk relasjon og tidslinje

Historie skal være en relasjonsmotor.

Ikke bare tekstfelt.

Relasjoner:

periode
dynasti / kongeslekt / maktstruktur
regent / konge / lokal hersker
territorium / område
objekt
personer
signaturer
historiske hendelser
materiale
funn
proveniens

I grafer og tabeller skal kongeperiode, dynasti og historisk periode kunne vises som kontekst under publiseringsår.

Det skal ikke erstatte publication_year.

Det skal ligge som historisk kontekst.

40. Admin

Admin er kontrollsenteret for hele plattformen.

Admin skal styre:

```
brukere
medlemskap
forhandlere
innleveringer
auksjoner
nettbutikk
katalog
objekter
AI/import
datakvalitet
betaling
avtaler
sider
features/brytere
API-ruter
logger
systemstatus
migrering
deploy readiness
```

Admin skal ikke være en løs backend med tilfeldige skjemaer.

Admin skal vise hele kjeden:

```
side
→ feature
→ tilgang
→ action-route
→ API
→ table/view
→ handling
→ logg
```

Admin skal kunne sjekke:

```
finnes siden i page registry?
finnes feature?
er side koblet til feature?
```

```
finnes access-regler?  
finnes action-route?  
finnes API-fil?  
finnes read_table/read_view/write_table?  
fungerer logging?  
finnes source_key/object_group?  
fungerer API JSON?  
finnes route i Next.js?  
finnes data i Neon?  
finnes data i MariaDB?
```

Admin skal ha "Svar til ChatGPT"-seksjon som kan kopieres inn for ny statusvurdering.

41. MariaDB → Neon Control

MariaDB → Neon Control er en egen admin/systemside:

```
/app/admin/system/mariadb-neon/page.tsx
```

Den skal ikke være en migreringsknapp.

Den skal være en kontrollflate som viser:

```
hva finnes i MariaDB  
hva finnes i Neon  
hva mangler i Neon  
hva er klart for struktur  
hva er klart for regler/prosesser  
hva er klart for kildedata  
hva er blokkert  
hva må godkjennes før migrering
```

Hovedregel:

1. Struktur først
2. Regler / metoder / prosesser etterpå
3. Kildedata til slutt

Dashboardet skal være totaloversikten.

Underfaner skal være detaljer.

Dashboardet skal vise:

```
total systemstatus
deploy gate
Neon truth status
migrering
sist kontrollert
sist menneskelig prosess
databaser
rammeverk
aktive moduler
API-ruter
DB-brytere/features
kataloger
relasjoner
Filter Master
brukeraktivitet
forbruk/kapasitet
diagnose
kritiske feil
varsler
mangler
neste tiltak
```

42. Neon namespace-regler

Neon-tabeller/views skal navngis med scope.

Låste prefikser:

```
ct_no_* = Norge
ct_sn_* = Skandinavia / nordisk relasjon
ct_sv_* = Sverige
ct_dm_* = Danmark
ct_tl_* = Tyskland
ct_fl_* = Finland
ct_eu_* = Europa
ct_gl_* = global/world
```

Eksempler:

```
ct_no_banknote_catalog
ct_no_coin_catalog
ct_sn_konger_relasjon
ct_sn_motiv_relasjon
ct_sv_banknote_catalog
ct_dm_coin_catalog
```

Objekter er ofte nasjonale.

Relasjoner kan være regionale, europeiske eller globale.

Kanoniske katalogtabeller:

```
ct_no_banknote_catalog
ct_no_coin_catalog
ct_sv_banknote_catalog
ct_sv_coin_catalog
```

Unscoped navn som `ct_coin_catalog` skal ikke være kanonisk for nasjonale kilder.

43. Import og staging

Nye katalogobjekter skal ikke skrives rett inn som sann katalogdata.

Ny seddel, ny mynt eller ny variant skal gjennom:

```
forslag / staging
→ kilde- og objektkontroll
→ relasjonskontroll
→ markeds-/verdi-kontroll
→ admin/forhandler-godkjenning
→ publisering i katalog
```

Tre tilfeller:

- A. Nytt eksemplar av kjent objekt
- B. Ny variant av kjent objekt
- C. Helt ny katalogpost

A. Nytt eksemplar av kjent objekt

Da skal ikke katalogposten dupliseres.

Brukerens konkrete eksemplar kobles til eksisterende katalogpost.

```
Katalogobjekt = objektdefinisjonen
Brukerobjekt = brukerens konkrete eksemplar
```

B. Ny variant av kjent objekt

Da skal den inn som forslag til ny katalogvariant.

For Norske sedler:

NSNR 23
= hovednummer / grunnpost

NSNR 23a
= variant under NSNR 23

NSNR 23b
= neste variant

NSNR 23c
= tredje variant

C. Helt ny katalogpost

Da skal den inn i staging/import/godkjenningsflyt.

Den skal ikke få endelig katalogstatus før kilde, relasjoner, bilder, variant og datakvalitet er kontrollert.

44. NSNR og variantnummer

For Norske sedler bør variantnummer håndteres slik:

NSNR 23	hovedpost / grunnvariant
NSNR 23a	variant A
NSNR 23b	variant B
NSNR 23c	variant C

Ikke:

NSNR 23
NSNR 24
NSNR 25

hvis 24 og 25 egentlig bare er varianter av 23.

Felt:

source_catalog_number
= full synlig katalogkode
Eksempel: NSNR 23a

source_catalog_base_number
= hovednummer

Eksempel: NSNR 23

source_catalog_variant_suffix
= variantbokstav

Eksempel: a

source_catalog_sort_number
= numerisk sortering

Eksempel: 23

source_catalog_sort_suffix
= variantrekkefølge

Eksempel: 1 for a, 2 for b, 3 for c

Sortering:

NSNR 21
NSNR 22
NSNR 23
NSNR 23a
NSNR 23b
NSNR 23c
NSNR 24

Teknisk sannhet er fortsatt:

object_id + object_group + source_key

Katalognummer er synlig kildeidentitet, ikke primær teknisk nøkkel.

45. API-regler

API-laget skal være mellomledet mellom Next.js/React og databasen.

Frontend skal ikke hente direkte fra database.

Eksempel for katalogsøk:

```
CatalogPage
→ getCatalogSearch(params)
→ /api/catalog/search
→ catalog objects resolved
→ catalog filter counts
```

```
→ user state
→ React viser resultat
```

Eksempel for brukerhandling:

```
WishlistButton
→ toggleWishlist(objectKey)
→ /api/collection/wishlist/toggle
→ user object state
→ logg
→ oppdatert user_state returneres
```

Filter API

```
GET /api/filter/master
GET /api/filter/options
POST /api/filter/resolve
GET /api/filter/period/simple
GET /api/filter/period/advanced
POST /api/filter/apply
```

Systemkontroll

```
GET /api/system/filter-master-check
GET /api/system/period-filter-check
GET /api/system/neon-health
GET /api/system/mariadb-health
GET /api/system/db-overview
GET /api/system/schema-inventory
GET /api/system/source-relation-overview
GET /api/system/table-mapping
GET /api/system/field-mapping
```

Objekt

```
GET /api/object/presentation
GET /api/object/relations
GET /api/object/market
GET /api/object/user-state
```

Relasjon

```
GET /api/relations/[relationType]/[relationKey]
GET /api/relations/[relationType]/[relationKey]/objects
GET /api/relations/[relationType]/[relationKey]/object-groups
```

```
GET /api/relations/[relationType]/[relationKey]/timeline
GET /api/relations/[relationType]/[relationKey]/linked-relations
GET /api/relations/[relationType]/[relationKey]/market
```

Index

```
GET /api/index/market
GET /api/index/period
GET /api/index/trends
GET /api/index/object-performance
GET /api/index/relation-performance
```

46. Auth / session

Innlogging og tilgang skal ikke være hardkodet.

Det skal styres av:

```
bruker
session
rolle
medlemskap
access-rule
feature access
action-route
logging
```

Auth-funksjoner:

```
auth.login
auth.session.create
auth.logout
auth.register
auth.email.verify
auth.password.reset
auth.membership.create
auth.session.view
auth.session.touch
```

Min side skal vise:

```
Logg inn / Registrer deg
```

hvis bruker ikke er innlogget.

Hvis bruker er innlogget, skal den vise:

```
Min side
Logg ut
rolleflate
medlemsstatus
varsler
meldinger
aktive prosesser
```

47. Sandbox og deploy-gate

Sandboxen er kontrollsystem før lansering.

Den skal teste:

```
filer
struktur
kode
template
skins
design
responsivitet
DB-kjede
API-ruter
kildegrunnlag
Collectium-regler
stabilitet
build
typecheck
lint
deploy-status
```

Brukerens rolle:

```
lese rapport
se OK / VARSEL / FEIL / KRITISK
planlegge neste steg
definere regler
justere krav
godkjenne eller avvise deploy
```

Systemets rolle:

```
kjøre kontroller  
finne feil  
teste filer  
teste build  
teste template/skins  
teste DB/API/kildekoblinger  
lage rapport  
blokkere deploy ved feil
```

Deploy-regel:

Deploy gate = ÅPEN

bare når:

```
root structure = OK  
rules/source = OK  
template = OK  
skins = OK  
build = OK  
typecheck = OK  
lint = OK eller godkjent varsel  
DB chain = OK eller godkjent varsel  
API = OK  
responsive = OK  
security = OK
```

Hvis ikke:

Deploy gate = BLOKKERT

48. Testtyper

Deploy UI-UX-DB-Git 8.5 skal vise testtype og kontrollnivå.

Testtyper:

```
Smoke test  
Health check  
End-to-end test  
Build test  
TypeScript test  
Lint test  
API test
```

```
DB chain test
UI test
UX test
Responsive test
Skin test
Security/secrets test
Recovery test
Deploy gate test
```

Smoke test

Sjekker om siden starter uten å krasje.

Health check

Sjekker app, API, database, miljøvariabler og serverstatus.

E2E-test

Tester faktiske brukerflyter:

```
login
katalog
filter
åpne objekt
hjerte/stjerne
legg i samling
auksjonsbud
adminstatus
```

49. Enhetstest / systemkontroll

Det skal finnes intern admin/systemside for enhetstest og kontroll.

Den skal kontrollere:

```
Next.js
React
API
MariaDB/Neon
DB-kjede
katalog source_key/object_group
bruker/medlemskap
forhandler
auksjon
objekt/relasjon
prosesslogg
```

```
recovery
regler
filstruktur
deploy readiness
```

Statusnivåer:

```
OK
VARSEL
FEIL
KRITISK
MANGLER
IKKE TESTET
```

Hver status skal være klikkbar og åpne detaljer.

50. Svar til ChatGPT

Alle større kontrollsider, rapporter og kodepakker skal ha en seksjon:

```
Svar til ChatGPT
```

Den skal inneholde:

```
Status
Hva er lagt til
Hva er endret
Hva er ikke rørt
Berørte filer
Berørte routes
Berørte feature_keys
Berørte API-ruter
Berørte DB-tabeller/views
Tester
Mangler
Neste anbefalte handling
Rollback
```

Dette gjør at status kan kopieres tilbake til ChatGPT for ny vurdering.

51. Byggerekkefølge

Riktig teknisk prioritet:

1. /admin/system/mariadb-neon
2. /admin/system/db84
3. /admin/system/filter-master
4. /admin/system/period-filter
5. /filter/master
6. /filter/periode
7. /filter/periode/avansert
8. /katalog
9. /objekt/[sourceKey]/[objectGroup]/[objectId]
10. /relasjon/[type]/[slug]
11. /index
12. /min-side
13. /samling
14. /auksjon
15. /nettbutikk
16. /forhandler
17. /admin

Filter Master og Periodefilter må bygges før katalog, objektpresentasjon, relasjonspresentasjon og index låses ferdig.

52. Hovedmoduler i Collectium

Collectium består av disse hovedmodulene:

1. System / kontroll / deploy
2. Auth / session / medlemskap
3. Global template / designmotor
4. Katalog
5. Filter Master
6. Periodefilter
7. Visningskort
8. Objektpresentasjon
9. Relasjonspresentasjon
10. Index / marked / finans
11. Min side
12. Samling
13. Ønskeliste / hjerte
14. Favoritter / stjerne
15. Transaksjoner
16. Proveniens
17. Forhandler
18. Auksjon
19. Nettbutikk
20. Museum / historie
21. Admin
22. Import / staging
23. Datakvalitet

- 24. Logging
- 25. Recovery
- 26. Sandbox

Alle modulene skal bruke samme grunnprinsipp:

Frontend viser.
API validerer.
Database eier sannhet.
Admin kontrollerer.
Logg dokumenterer.
Sandbox godkjenner før deploy.

53. Endelig låst modell

Visningskort

- = kompakt objektingang
- = viser objekt, verdi, år, periode og raske relasjoner
- = åpner objekt, relasjon eller samlingshandling

Objektpresentasjon

- = full visning av ett objekt
- = viser Samler, Historie og Finans
- = bruker periode og relasjon som kontekst
- = har fane for relasjon til andre objekttyper

Relasjonsside

- = kunnskapsside for én node
- = viser objekter, periode, historikk, marked og videre relasjoner
- = har fane for relasjon til andre samleobjekter

Index

- = analyse- og markedsmotor
- = eier bred periodeanalyse og utvikling over tid
- = bruker katalog, relasjoner, samling, auksjon, nettbutikk og prisobservasjoner

Periodefilter

- = felles tidsmotor
- = brukes på katalog, objekt, relasjon, index, auksjon, forhandler og samling

Filter Master

- = felles filterkontrakt
- = hindrer at hver side lager sin egen filterlogikk

Admin/system

- = kontrollsenter

= viser side → feature → access → API → table/view → logg

Neon

= ny hoveddatabase når kontroll er OK

MariaDB

= kilde og kontrollarkiv i overgangsperioden

Vercel

= app, API-runtime, deploy og miljøvariabler

GitHub

= kodebase, ikke hemmeligheter

Kort läsbar formulering:

Collectium skal bygges som en kontrollert Next.js/React-basert relasjonsplattform der visningskort, objektpresentasjon, relasjonsside og index deler samme Filter Master og periodefilter. Visningskortet viser kort objektkontekst, objektpresentasjonen forklarer ett objekt, relasjonssiden forklarer én relasjonsnode, og index analyserer utvikling på tvers av perioder, marked og objekter. Frontend skal ikke finne opp relasjoner, perioder eller filterverdier; API/database skal levere ferdige nøkler, href-er, tilgang, status og logg.

54. Kort konklusjon

Collectium har nå fått en tydeligere og sterkere retning:

Fra:

løse sider, lokal design, manuelle reparasjoner og isolerte visninger

Til:

kontrollert Next.js/React-plattform med global template, DB/API-sannhet, Neon-overgang, Filter Master, periodefilter, relasjonspresentasjon, sandbox, deploy-gate og rollebaserte moduler.

Den viktigste regelen fremover er at Collectium ikke må bygges som raske enkeltfiler som bare “ser riktige ut”.

Hver ny side, knapp, filter, modul og prosess må kobles til:

riktig kontrollkjede
riktig datakilde
riktig feature

```
riktig API  
riktig tilgang  
riktig logg  
riktig relasjon  
riktig periodekontekst
```

Dette er det som gjør at Collectium kan vokse til en stor katalog-, samler-, museum-, auksjons- og markedsplattform uten å miste kontroll.